Progressive Visibility Computation and Heuristic Optimization for Antenna Placement

Author preprint · Version 1 · November 5, 2026

Xuezhi Cang
Discovery Partners Institute, University of Illinois Urbana-Champaign
Chicago, USA
xuezhicang@gmail.com

Abstract

Selecting antenna locations on building boundaries requires accounting for both geometric occlusion and the combined coverage of multiple antennas. This paper presents the solution submitted to the 15th ACM SIGSPATIAL GIS Cup, 2026, consisting of two stages: progressive visibility computation and antenna placement optimization. In the first stage, the method samples candidate points along building boundaries and precomputes their visible boundary intervals using a spatial index. For each candidate location, progressive visibility computation expands a local search region in stages, using angular sweeps to identify visible boundary intervals and prune blocked viewing directions from subsequent searches. In the second stage, standard greedy search and a two-step lookahead variant generate initial solutions, which are refined through one-point swaps and simulated annealing. Reusable visibility intervals support efficient heuristic search across different antenna budgets and coverage thresholds.

CCS Concepts

• **Information systems → Geographic information systems**.

Keywords

antenna placement, boundary visibility, greedy search, simulated annealing, GIS Cup

1 Introduction

The 15th ACM SIGSPATIAL GIS Cup (2026) challenge asks participants to place antennas on building boundaries to maximize the number of buildings meeting a required visible-perimeter fraction [1]. The task couples geometric visibility with location selection: buildings obstruct sight lines, while multiple antennas may observe overlapping or complementary portions of the same building boundary. Consequently, the value of an antenna location depends on the other locations selected.

Two aspects make this problem challenging. First, evaluating visibility across a large collection of building footprints requires accounting for occlusion by surrounding buildings. Repeating these geometric calculations during placement search would be costly. Second, the service objective is threshold-based: additional visible boundary contributes to the score only when a building reaches the required coverage fraction. A candidate with little immediate benefit may therefore become valuable when combined with other antennas, making selection based only on immediate gains insufficient.

Heuristic search over the sampled candidate set can explore complementary placements, and simulated annealing can help escape local optima. However, these methods may require many solution evaluations and provide no guarantee of global optimality within a finite computational budget. Efficient geometric computation and reuse of visibility results are therefore important foundations for the search.

This paper focuses on accelerating boundary visibility computation through spatial indexing and parallel execution on CPUs. The spatial index restricts the geometry processed by each angular sweep, while independent visibility queries for different candidate locations are distributed across MPI ranks. The resulting boundary intervals are precomputed and reused throughout placement optimization, allowing the search to evaluate combined coverage without repeating geometric visibility calculations. This design separates geometric preprocessing from heuristic search and allows the same visibility data to support all nine competition settings. Figure 1 summarizes the full workflow, including visibility preprocessing, reusable coverage scoring, and placement optimization.

2 Problem Definition and Dataset

The task is to select k antenna locations on building boundaries to maximize the number of serviced buildings [1]. For an antenna set S, the coverage $c_b(S)$ of building b is the fraction of its perimeter visible from at least one selected antenna; overlapping visible portions are counted only once. A building is serviced when $c_b(S) \geq \tau$, where τ is the required coverage threshold. The objective $H_\tau(S)$ counts the serviced buildings. Visibility follows the competition rules: sight lines must not cross building interiors, while boundary tangency alone does not block visibility. For a given antenna budget k and coverage threshold τ, a solution S with a higher service count $H_\tau(S)$ is preferred.

The dataset contains 50,000 building footprints in an approximately 10 km×10 km area. Each footprint is represented by a simple polygon. The competition uses antenna budgets $k \in \{9, 49, 484\}$ and coverage thresholds $\tau \in \{0.32, 0.49, 0.68\}$.

3 Visibility Preprocessing

3.1 Discrete candidate locations

In this study, all building vertices are retained as the baseline candidate set. Additional points are sampled at intervals of 5 m along

This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).

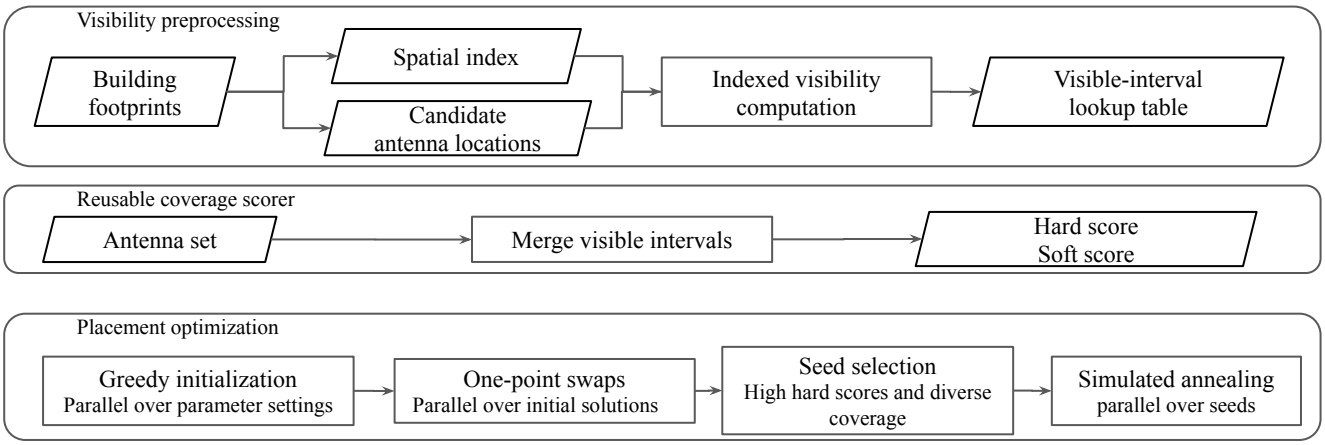


Figure 1: Overview of visibility preprocessing, reusable coverage scoring, and placement optimization.

each polygon edge, measured from its starting vertex. This procedure preserves building corners and ensures that consecutive candidate points along each edge are no more than 5 m apart.

Denser sampling provides more placement options but increases both visibility preprocessing costs and the size of the placement search space. This discretization keeps candidate locations on building boundaries, but does not guarantee that the continuous optimum is included in the candidate set.

3.2 Angular-sweep visibility

The angular-sweep visibility computation is based on the visibility algorithms described by O'Rourke [3, Chapter 8]. For a fixed antenna, the basic visibility operation rotates a ray around that point. Building-edge endpoints define angular events at which edges enter or leave the active set. Between events, the implementation finds the nearest positive ray–edge intersection among active edges and checks the frontmost edge before recording a visible segment. Intersections near the angular interval endpoints determine the corresponding portion of that edge. Its endpoints are stored as normalized parameters $[s_1, s_2] \subseteq [0, 1]$ on the original polygon edge.

The antenna's own building requires explicit handling because the viewpoint lies on its boundary. The implementation checks whether sight lines to that building cross its interior and adds visible edges of that building separately. Segments are merged after the sweep. The calculation uses floating-point geometry and numerical tolerances. Applying this operation to every building edge for every candidate would repeat substantial work; the spatial index narrows the geometry supplied to each sweep.

3.3 Accelerating the sweep with a spatial index

In this implementation, the nearest intersected edge is identified by scanning the active edges at each angular event. For a sweep involving n building edges, these repeated searches can require $O(n^2)$ work in the worst case. Applying this procedure to all building edges for each of the 812,994 candidate locations would be computationally demanding. To reduce the geometry processed by each sweep, the method builds a spatial index that partitions buildings according to their centroids. Each node stores an expanded bounding box enclosing its assigned building footprints. Queries retrieve buildings from nodes whose boxes intersect the current search region. Combined with pruning of viewing directions already blocked by nearby buildings, this approach can avoid processing distant geometry that cannot contribute additional visible boundary segments.

Index construction. The visibility engine builds a quadtree-based spatial index over building centroids. At each node, the midpoints of the centroid coordinate ranges define horizontal and vertical split lines. Buildings are assigned to the resulting nonempty quadrants and partitioned recursively. Subdivision stops when a node contains at most 64 buildings or when the centroids cannot be separated into multiple children. Node memberships occupy contiguous ranges in an array of building identifiers.

Centroids determine membership, but complete footprints determine query bounds. Each node's partition cell is expanded to enclose the axis-aligned bounding boxes of its assigned buildings. These expanded boxes may overlap. A building crossing a partition line is therefore still included in intersection queries even if its centroid lies outside the query region. Tree traversal prunes nodes whose expanded boxes do not intersect the query box. A nearest-boundary query also uses point-to-box distance bounds to prioritize nodes and restrict polygon-edge tests.

Spatial expansion and angular pruning. For each antenna, the search starts from the combined extent of all leaf boxes containing its snapped boundary location, with the full angular range $[0, 2\pi)$ initially unresolved. At each iteration, the index identifies leaf boxes intersecting a small buffer around the current search box. The search box is then enlarged to enclose both the buffered region and these complete leaf boxes. A second index query retrieves all leaves intersecting the enlarged box. An unvisited leaf contributes buildings only if the angular extent of its box, as seen from the antenna, overlaps the remaining unresolved directions.

After all eligible leaves have been selected, the loaded building edges are clipped to the current search box and processed together

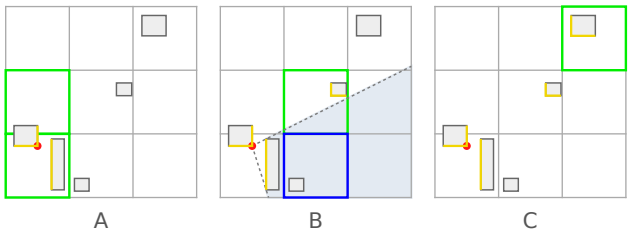


Figure 2: Spatial expansion and angular pruning. Red: antenna; green: selected cells; blue: fully occluded, pruned cell; yellow: cumulative visible boundaries. Shading and dashed rays indicate the occlusion region.

by an angular sweep. The sweep records visible boundary segments and identifies directions whose nearest blocking edges have been determined. These directions are removed only after the batch is complete, and the remaining directions guide the next round of node selection. Previously loaded buildings are reconsidered as the box expands, allowing edge portions outside the earlier search region to enter the calculation. Expansion continues until no unresolved directions remain, the search box covers the dataset bounds, or the iteration limit is reached.

Thus, spatial intersection determines which nodes are considered, while overlap with unresolved viewing directions determines which nodes supply buildings to the angular sweep. Figure 2 illustrates this process schematically.

This combination restricts both where geometry is loaded and which directions still require examination. It reduces unnecessary sweep input when nearby buildings already block distant ones; its benefit depends on the layout and occlusion pattern. MPI distributes candidate queries across ranks. The output records candidate, building, and edge identifiers, interval endpoints, edge lengths, and perimeters. This table is independent of k and τ and is shared by all subsequent searches.

4 Antenna Placement Optimization

4.1 Shared coverage scoring

All three optimization stages share an interval-union scorer. Each visible boundary segment is represented by a normalized interval $[s_1, s_2] \subseteq [0, 1]$ on its building edge, where 0 and 1 denote the edge's starting and ending vertices. The scorer stores a sorted list of disjoint coverage intervals for each edge in a hash table, using a composite key containing the building identifier and local edge index. A hash-table lookup retrieves the existing coverage of the corresponding edge without scanning all building and edge identifiers.

For each candidate antenna, the scorer computes the additional normalized interval length beyond the existing coverage on each edge. These gains are converted to physical lengths using the corresponding edge lengths and summed per building. When the candidate is selected, a two-pointer scan merges its sorted intervals with the existing union, combining overlapping or touching intervals, and updates the stored visible lengths. Dividing each building's total visible boundary length by its perimeter gives its coverage $c_b(S)$. This interval representation supports incremental coverage evaluation without repeating geometric visibility calculations during search.

The hard score $H_\tau(S)$ changes only when a building crosses the service threshold, so an increase in coverage may leave this score unchanged. To reward progress toward the threshold, the search also uses the capped soft score

$$F_\tau(S) = \sum_{b \in \mathcal{B}} \min\{c_b(S)/\tau, 1\}, \tag{1}$$

where $\mathcal{B}$ denotes the set of buildings.

Each building contributes $c_b(S)/\tau$, capped at 1, so coverage beyond the service threshold gives no additional reward. The soft score therefore rewards partial progress even when the hard score remains unchanged.

4.2 Greedy initialization

Greedy initialization constructs antenna sets for subsequent swap refinement and simulated annealing. In standard greedy search, each iteration selects the antenna with the highest evaluation score. Early selections can limit the quality of the final configuration, particularly when candidate locations provide complementary coverage. To account for this interaction, the method combines hard and soft coverage gains to guide candidate selection and uses two-step lookahead to evaluate the combined contribution of successive additions. This helps identify antenna locations whose value may not be apparent from their immediate hard-score gains.

For an unselected candidate p, let $\Delta H(p)$ and $\Delta F(p)$ denote marginal hard and soft gains. Greedy ranking combines their normalized values:

$$G_\alpha(p) = \alpha \frac{\Delta H(p)}{\max_q \Delta H(q)} + (1-\alpha) \frac{\Delta F(p)}{\max_q \Delta F(q)}, \tag{2}$$

where q ranges over all unselected candidate locations. A term is defined as zero when its denominator is zero. Hard gain favors immediately serviced buildings, while soft gain also values useful partial coverage. The search explores $\alpha \in \{0.4, 0.6, 0.8, 1.0\}$.

Two-step lookahead evaluates the combined gain of two successive antenna additions before choosing which antenna to add next. The lookahead width w specifies the number of candidates retained at each stage. The algorithm first shortlists the top w candidates as possible first choices. For each first choice, it copies the current state, tentatively adds that antenna, and rescores all remaining candidates to retain the top w second choices.

Each resulting pair is evaluated using its cumulative hard and soft gains. These gains are normalized separately across the evaluated pairs and combined using weight α. Only the first antenna of the best pair is added to the current solution, and scores are recomputed before the next selection. Because each tentative first choice requires rescoring the remaining candidates, wider lookahead increases computational cost. This paper evaluates lookahead widths $w \in \{1, 5, 10\}$.

4.3 One-point swap refinement

One-point swap refinement improves antenna locations while keeping the antenna count fixed. For each current solution, the algorithm

temporarily removes each selected antenna in turn, keeping the others in place. It then evaluates all unselected candidates and greedily chooses the replacement that yields the highest number of serviced buildings. Only swaps that improve the corresponding original solution are accepted, and a small number of the highest-scoring distinct solutions are retained for the next round. The process continues until no further improving swaps are found or a preset number of rounds is reached.

One-point swapping can stall when improvement requires changing several locations together or accepting a temporary score decrease. To initialize simulated annealing, five solutions per setting are selected from the swap results based on both their scores and the diversity of their serviced-building sets.

4.4 Simulated annealing

Simulated annealing [2] refines the swap solutions by accepting temporary score decreases to escape local optima. It explores new configurations by randomly removing one or more antennas and selecting replacements sequentially from sampled candidate pools. Candidates are ranked by hard gain, soft gain, and newly covered boundary length, in that order. Replacement selection favors higher-ranked candidates while retaining a probability of random selection from the entire pool. Moves that increase the number of serviced buildings are always accepted. Worse moves may also be accepted, with a higher probability at high temperatures and a lower probability as the search cools. When the hard score remains unchanged, the soft score guides acceptance.

Independent parallel searches use different initial solutions, random seeds, and temperature profiles. Each run follows a cooling schedule, retains its best solution, and can restart from it with reheating. The final result has the highest hard score across all runs.

OpenAI Codex was used as a programming assistant to implement and debug portions of the competition code. The author designed the algorithms, reviewed and revised all AI-assisted code, executed the experiments, and independently verified the reported results.

5 Results and Discussion

All computations were performed on the Metis high-performance computing cluster at Northern Illinois University (NIU). Sampling the boundaries of the 50,000 building footprints produced 812,994 candidate locations. Visibility preprocessing, including data preparation and output, took 41.4 minutes using 300 CPU cores. This preprocessing generated 60,209,926 visible boundary interval records, averaging approximately 74.1 intervals per candidate location.

Antenna placement optimization began after the precomputed intervals were loaded. As a runtime baseline, standard greedy search without lookahead took approximately 2 seconds per step on a single CPU core to evaluate all remaining candidates, select one antenna location, and update coverage, as estimated from the $k = 9$ multistart runs. Two-step lookahead with width w requires one initial pass over the candidates and w additional passes, one for each tentative first choice. Assuming a similar cost per pass, this gives a rough estimate of $2(1 + w)$ seconds per step, or approximately 12 seconds per antenna added when $w = 5$.

Table 1: Best locally reported serviced-building counts at each search stage in the competition workflow. Parentheses show gains over the preceding stage's best. These are not organizer-verified scores.

τ	k	Greedy	1-p swap	Final
0.32	9	1,401	1,403 (+2)	1,403 (+0)
0.32	49	4,007	4,035 (+28)	4,040 (+5)
0.32	484	15,880	16,020 (+140)	16,046 (+26)
0.49	9	814	818 (+4)	818 (+0)
0.49	49	2,499	2,532 (+33)	2,538 (+6)
0.49	484	10,369	10,483 (+114)	10,532 (+49)
0.68	9	255	258 (+3)	260 (+2)
0.68	49	1,103	1,170 (+67)	1,182 (+12)
0.68	484	5,562	5,681 (+119)	5,763 (+82)

Table 1 compares the best serviced-building counts at each search stage. Greedy initialization provides strong starting solutions, while one-point swaps and simulated annealing yield further improvements. The larger absolute gains at higher antenna budgets may reflect more opportunities to improve combined coverage by adjusting the selected locations.

6 Conclusion

This paper presents an antenna placement pipeline that combines spatially indexed visibility preprocessing with greedy initialization, one-point swaps, and simulated annealing. By reusing precomputed boundary intervals, the search evaluates coverage without repeating geometric calculations. Swaps and annealing improve the greedy solutions, with larger absolute gains at higher antenna budgets. Future work includes adaptive boundary sampling and controlled evaluation of preprocessing and search costs.

Acknowledgments

The author thanks Prof. Wei Luo for providing helpful feedback, and the High Performance Computing Center at Northern Illinois University for providing computational resources. OpenAI Codex was used to assist with language editing of the manuscript.

References

[1] ACM SIGSPATIAL GIS Cup Organizers. 2026. 2026 GIS Cup: Antenna Placement and Building Coverage. https://sigspatial2026.sigspatial.org/giscup.html

[2] S. Kirkpatrick, C. D. Gelatt, Jr., and M. P. Vecchi. 1983. Optimization by Simulated Annealing. *Science* 220, 4598 (1983), 671–680. doi:10.1126/science.220.4598.671

[3] Joseph O'Rourke. 1987. *Art Gallery Theorems and Algorithms.* Oxford University Press, New York, NY, USA. https://www.science.smith.edu/~jorourke/books/ArtGalleryTheorems/art.html